

Good Liquid Bev Co CRM

AI-Assisted Engineering, Architecture & Code Review Standard

Version 2.0 - August 2026

1. Purpose

- This document establishes the engineering standards and AI collaboration workflow for the existing Good Liquid Bev Co CRM. The CRM is already a live, working application. The purpose is not to rebuild it; it is to protect the existing system while allowing future enhancements to be developed safely.
- The primary problem this standard addresses is regression: a new enhancement can unintentionally break an unrelated feature, introduce tight coupling, create security weaknesses, or store important information in browser cache/local storage instead of the authoritative database.

2. Current Technology & Operating Model

- Source control: GitHub.
- Database/backend platform: Supabase.
- Primary development agent: Claude Code, working against the local development environment and GitHub repository.
- Independent reviewer: ChatGPT/Codex, used as a second set of eyes for architecture, code quality, security, database design, testing, regressions, and enhancement opportunities.
- Production system: live CRM. Production changes must therefore be treated as controlled releases, not casual edits.

3. Core Architecture Principle: Modular by Design

- Every major business capability must be implemented as a self-contained module. Examples include Invoicing, Sales Pipeline, Customers/CRM, Products, Quotes, Orders, Production, Inventory, Scheduling, Reporting, Users/Roles, and Notifications.
- Modules must communicate through explicit interfaces, shared service layers, or well-defined APIs rather than reaching directly into another module's internal implementation.
- A change to one module should require minimal or zero modification to unrelated modules. Shared code belongs in clearly defined common services/utilities, not copied throughout the application.
- Do not create hidden dependencies between modules. Avoid global state, undocumented side effects, circular imports, direct manipulation of another module's UI state, and database queries that silently couple unrelated business functions.

4. Database Is the System of Record

- Business-critical data belongs in Supabase/database tables, not browser cache, localStorage, sessionStorage, IndexedDB, cookies, or temporary client state.
- Browser-side state may be used for UI convenience, drafts, temporary filters, performance caching, and non-authoritative session information, but it must never become the only copy of business data.
- Before adding a new persistent field, Claude must determine whether the data belongs in an existing table or requires a new normalized table/relationship.

- Database changes must include appropriate constraints, indexes, foreign keys, defaults, validation, and Row Level Security (RLS) policies where applicable.
- Any migration must be reversible or have a documented rollback strategy.

5. Supabase Standards

- Use Supabase as the authoritative persistence layer for CRM business data.
- Review table design for normalization, referential integrity, appropriate data types, indexes, constraints, and RLS.
- Never expose service-role credentials or other privileged secrets to browser/client code.
- Secrets and credentials belong in environment variables or the appropriate secret-management mechanism.
- Database access should go through predictable service/repository functions where practical rather than scattered ad-hoc queries throughout UI components.

6. GitHub Workflow & Branch Protection

- Claude should develop enhancements on a feature branch rather than directly modifying the production branch.
- Each meaningful enhancement should have a focused commit history and a pull request (PR) describing what changed, why it changed, affected modules, database migrations, tests, and potential risks.
- ChatGPT/Codex reviews the PR/diff and, when access is available, the relevant repository structure and database migrations.
- No feature should be merged solely because it works on the happy path. It must also pass regression, security, data-integrity, and module-boundary review.
- Production deployment should occur only after the review gate is satisfied and the owner approves the release.

7. AI Collaboration: Claude as Developer, ChatGPT as Independent Reviewer

- Claude is the primary implementation agent. Claude owns feature development, local testing, debugging, and preparing the GitHub PR.
- ChatGPT/Codex is the independent second set of eyes. It should not simply rubber-stamp Claude's work. It should actively look for defects, regressions, security problems, weak architecture, poor database practices, unnecessary complexity, and opportunities to improve the design.
- The two AIs should operate as independent reviewers rather than one AI blindly trusting the other's conclusions.

8. Review Gate and Revision Loop

- Step 1 — Define the enhancement: identify the business objective, affected module(s), expected behavior, data changes, permissions, and acceptance criteria.
- Step 2 — Claude implements the change on a feature branch.
- Step 3 — Claude runs automated tests, linting, type checks, and relevant local/integration tests.
- Step 4 — Claude opens a GitHub PR with a concise change summary and test evidence.
- Step 5 — ChatGPT reviews the diff and surrounding architecture. Review includes regression risk, module boundaries, database persistence, Supabase/RLS security, authentication/authorization, input validation, error handling, performance, maintainability, and test coverage.

- Step 6 — ChatGPT produces findings with severity: BLOCKER, HIGH, MEDIUM, LOW, or INFORMATIONAL.
- Step 7 — BLOCKER/HIGH findings must be fixed before approval. Claude revises the branch and updates the PR.
- Step 8 — ChatGPT re-reviews the changes. The cycle repeats until the review gate passes.
- Step 9 — Owner gives final approval and the change is merged/deployed.

9. Regression Protection

- Before modifying an existing feature, identify its current behavior and dependencies.
- Enhancements must not silently alter unrelated workflows, database records, permissions, calculations, reports, navigation, or UI behavior.
- Prefer automated regression tests for critical workflows. At minimum, test the changed module plus adjacent workflows likely to be affected.
- Use database migrations and deployment previews/staging where available before touching production.
- After deployment, verify the changed workflow and critical adjacent workflows. Monitor logs/errors for unexpected behavior.

10. Security Review Standard

- Review authentication and authorization on every feature that handles customer, employee, financial, operational, or administrative data.
- Verify Supabase RLS policies and ensure users cannot read or modify records outside their authorized scope.
- Check for SQL injection, XSS, CSRF where applicable, insecure direct object references, broken access control, exposed secrets, unsafe file uploads, weak validation, insecure API endpoints, and sensitive information in logs.
- Never trust client-side permission checks. Authorization must be enforced server-side/database-side.
- Review dependencies for known vulnerabilities and avoid unnecessary packages.

11. Code Quality Standards

- Favor readable, explicit code over clever shortcuts.
- Do not duplicate business logic when it can be safely centralized.
- Keep components/functions reasonably small and focused.
- Use consistent naming, typing, error handling, logging, and API conventions.
- Do not introduce temporary hacks without documenting them and adding them to the technical-debt register.
- Do not use browser storage as a shortcut to avoid designing the proper database schema.

12. Technical Debt Register

- Maintain a living list of known shortcuts, duplicated logic, architectural weaknesses, missing tests, security concerns, and deferred refactors.
- Each item should include severity, affected module, business impact, recommended fix, and status.
- Technical debt should be reviewed during major enhancements so the system does not accumulate hidden fragility.

- Principle: refactor before adding complexity when the existing architecture makes a requested enhancement unnecessarily risky.

13. Enhancement Review Beyond the Requested Feature

- ChatGPT's review should include a second question: 'Is there a better way to build this?'
- Reviewers should identify missing capabilities, unnecessary complexity, opportunities to consolidate duplicated code, better database relationships, improved workflows, automation opportunities, reporting gaps, performance improvements, and security hardening.
- Recommendations should be separated from mandatory fixes so optional improvements do not unnecessarily block a release.

14. Suggested Automated Review Cadence

- Every feature PR: targeted code, architecture, security, database, and regression review.
- Daily or continuous automated checks where practical: tests, linting, type checking, dependency/security scanning, and build validation.
- Weekly deep review: scan the repository for architectural drift, duplicated code, browser-only persistence, security weaknesses, dead code, poor module boundaries, database inconsistencies, and accumulating technical debt.
- Monthly architecture review: examine whether the CRM is still modular, scalable, secure, and maintainable as new modules are added.

15. Standard PR Review Checklist

- Business requirement and acceptance criteria are clear.
- Correct module boundary identified.
- No unnecessary cross-module coupling introduced.
- Business data persists in Supabase/database.
- No sensitive data stored in browser storage.
- Database migration reviewed.
- RLS/authentication/authorization reviewed.
- Input validation and error handling reviewed.
- Secrets are not exposed.
- Automated tests added/updated where appropriate.
- Existing workflows/regressions considered.
- Performance impact considered.
- Dependencies reviewed.
- Rollback path understood.
- Technical debt created by the change is documented.

16. Definition of Done

- The feature works as intended.
- The affected module remains independently maintainable.
- Critical data is persisted correctly in Supabase.
- Security and authorization controls are verified.
- Relevant tests pass.

- Regression risk has been reviewed.
- ChatGPT/Codex review findings are resolved or explicitly accepted by the owner.
- GitHub PR is approved.
- Production deployment is controlled and verified.

17. Long-Term Goal

- The goal is not simply to make the CRM functional. The goal is to make it increasingly difficult for a future enhancement to break something that already works.
- The architecture should therefore favor modularity, explicit interfaces, authoritative database persistence, automated testing, controlled GitHub releases, and independent AI review.
- Claude should be optimized for productive implementation. ChatGPT should be optimized for skepticism: challenge assumptions, inspect the surrounding system, identify hidden dependencies, and look for safer or cleaner alternatives.
- Human ownership remains the final authority for production releases and business decisions.

Operating Rule

If a proposed enhancement cannot be made without risking unrelated working functionality, stop and redesign the change before deployment. The default answer to architectural uncertainty is to isolate the change, persist authoritative data in Supabase, test the affected boundaries, and obtain independent review.

Version 2.0 Addendum - Automated Review and Remediation

The following setup instructions are part of the engineering standard and define how the independent AI review process should be implemented operationally.

Automated AI Review Infrastructure - Setup Instruction #1

- Objective: create a repeatable GitHub-centered pipeline so code can be reviewed without manually copying large amounts of code between Claude and ChatGPT.
- GitHub remains the source of truth for application source code, branches, pull requests, review history, and release status.
- Claude remains the primary developer. Claude creates a feature branch, implements the enhancement, runs local tests, and opens or updates the pull request.
- Configure an automated scheduled workflow for a deep repository review every Tuesday at 8:00 AM. The implementation must explicitly configure the intended timezone rather than assuming UTC.
- The scheduled review should collect the current repository state, changes since the previous review, open pull requests, database migrations, dependency/security scan results, and test/build status.
- ChatGPT/Codex should review the relevant code and produce both a human-readable report and structured findings containing: finding ID, severity, affected module/files, explanation, recommended remediation, and release-blocking status.
- Review results must be written to a predictable GitHub location - for example a GitHub issue, pull-request review/comment, workflow artifact, or dedicated review record - so Claude can consume the findings without relying on conversational memory.
- Use GitHub Actions secrets/environment variables or an equivalent secrets manager for credentials. Never commit API keys, Supabase service-role keys, tokens, or other secrets to the repository.
- The weekly review is supplemental to pull-request review. High-risk changes should be reviewed before merge rather than waiting for the Tuesday audit.

Automated AI Remediation Infrastructure - Setup Instruction #2

- Objective: create a controlled feedback loop in which ChatGPT/Codex findings become actionable remediation work for Claude and are independently re-verified.
- Each review finding receives a unique identifier and severity classification: BLOCKER, HIGH, MEDIUM, LOW, or INFORMATIONAL.
- BLOCKER and HIGH findings must create or update an actionable GitHub issue or pull-request review item and must prevent release unless the owner explicitly accepts the risk.
- Claude reads the review findings, inspects the relevant surrounding code, corrects the underlying defect on the existing feature branch or a dedicated remediation branch, and adds or updates tests that demonstrate the correction.
- Claude must not merely suppress, delete, bypass, or rewrite a finding to make the review pass. The underlying problem must be fixed or explicitly accepted by the owner.
- After Claude pushes the remediation commit, the review workflow runs again against the updated code.
- The re-review must explicitly mark every original finding as RESOLVED, STILL PRESENT, or ACCEPTED BY OWNER and should identify any new regression introduced by the remediation.
- The remediation/re-review loop continues until all release-blocking findings are resolved or explicitly accepted by the owner.
- The final pull request should retain the implementation summary, test evidence, review findings, remediation evidence, and final approval status so there is an auditable history.

- Automatic production deployment must be blocked when the review gate fails. A successful build alone is not sufficient approval for production.

Implementation Notes for Claude

- When this standard is handed to Claude, Claude should first audit the existing CRM against it before changing the architecture. The audit should identify where the current code already complies and where technical debt or risk exists.
- Claude should propose the exact GitHub automation implementation needed for this repository, including workflow files, required secrets, branch-protection settings, status checks, review-output location, and the mechanism by which Claude will consume remediation findings.
- Before enabling automated remediation against a live system, test the workflow on a non-production branch and confirm that it cannot merge or deploy changes without the required review gate.
- The automation should augment normal engineering controls - tests, branch protection, dependency scanning, Supabase security, backups, staging/preview environments, and human release authority - rather than treating either AI as a security boundary.

Updated Operating Rule

If an enhancement can break unrelated working functionality, stop and redesign the change before deployment. Isolate the change, persist authoritative business data in Supabase, test affected boundaries, run independent review, remediate release-blocking findings, re-review the corrections, and only then approve release.